Q1: Retriever & Reranker Tuning

Training data construction & Loss function

1. I convert the train.txt to train.json by data2json.py, making it a list of dict.
2. The training data for both finetuning retriever and reranker is constructed with the “rewrite” value as anchor/query, and the passage in the “evidences” with “retrieval_labels” equal to 1 as positive/answer in each example of train.json.
3. For finetuning retriever, I use Multiple Negatives Ranking Loss, which will sample in each batch n-1 negative documents randomly, and I train the retriever with full dataset for 1 epoch.
4. For finetuning reranker, I use mine_hard_negatives() imported from sentence_transformers.util to add hard negatives to the dataset, which can improve the performance of models trained on the dataset. I train the reranker for 1 epoch with full dataset, each example with 4 randomly mined negatives, and I use Cached Multiple Negatives Ranking Loss as loss function, which is a boosted version of MultipleNegativesRankingLoss that caches the gradients of the logits w.r.t. the loss and allows for much higher batch sizes without extra memory usage.

Hyperparameters (Figure 1: retriever, Figure 2: reranker)

```
args = SentenceTransformerTrainingArguments(
    num_train_epochs=n_epoch,
    per_device_train_batch_size=64,
    per_device_eval_batch_size=64,
    learning_rate=5e-5,
    warmup_ratio=0.1,
    bf16=True,
    batch_sampler=BatchSamplers.NO_DUPLICATES,
    logging_steps=50,
)
```

Figure 1

```
train_batch_size = 128
num_epochs = 1
num_rand_negatives = 4
dataset = mine_hard_negatives(
    dataset=train_dataset,
    model=retriever,
    range_min=10,
    range_max=50,
    max_score=0.8,
    relative_margin=0.05,
    num_negatives=num_rand_negatives,
    sampling_strategy="random",
    batch_size=256,
    use_faiss=True,
)
loss = CachedMultipleNegativesRankingLoss(
    model=model,
    num_negatives=num_rand_negatives,
    mini_batch_size=64,
)
args = CrossEncoderTrainingArguments(
    num_train_epochs=num_epochs,
    per_device_train_batch_size=train_batch_size,
    per_device_eval_batch_size=train_batch_size,
    learning_rate=2e-5,
    warmup_ratio=0.1,
    fp16=False,
    bf16=True,
    eval_strategy="no",
    save_strategy="steps",
    save_steps=100,
    save_total_limit=2,
    logging_steps=50,
    logging_first_step=True,
)
```

Figure 2

Q1: Retriever & Reranker Tuning

Training loss curve (Figure 3: retriever, Figure 4: reranker)

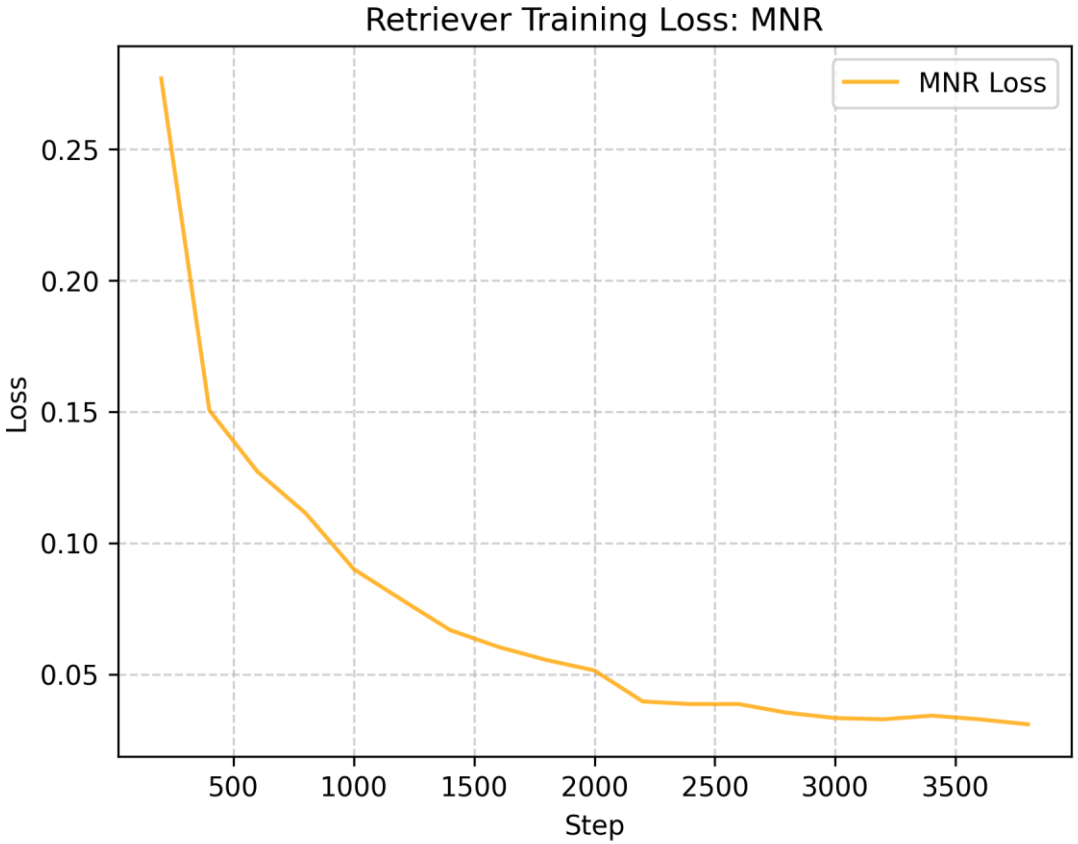

Figure 3

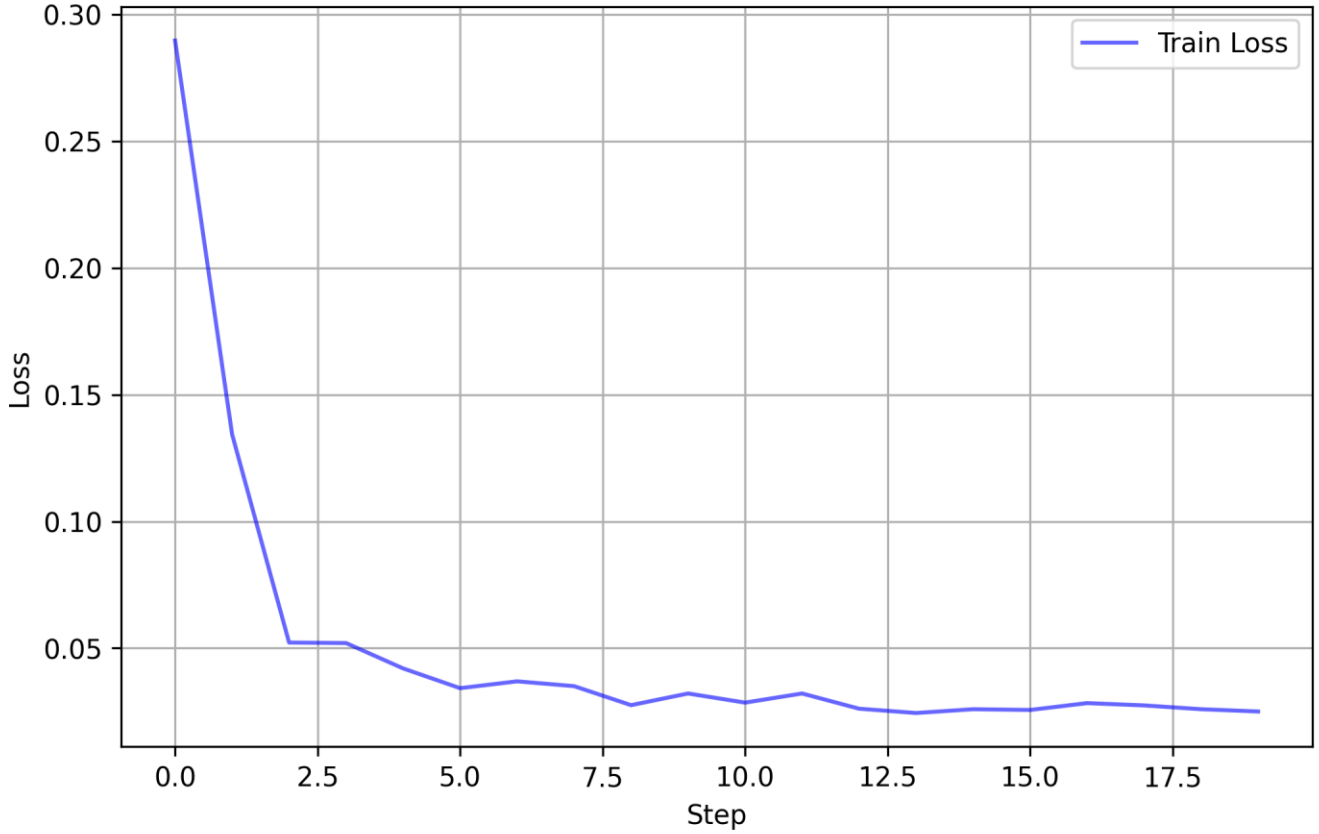

Figure 4

Q2: Prompt Optimization

- I try to keep the prompt as simple as possible while explicitly tell the LLM its role, task, and decision to make if there's no answer in the given passages.
- I tried 3 different prompts. The performance (tested on the first 256 examples) and details are shown in order in Figure 5, 6, 7, 8.
- V1: Add simple instructions about role, task, and decision to system and user prompts. No parsing generated answer. This turns out reaches best performance.
- Base: No system prompt. Just layout the query and passages. No parsing generated answer.
- V2: Strongly prevent hallucination in the 3 functions, which may lead to more "CANNOTANSWER" and therefore lower the Bi-Encoder Cosine Similarity (CosSim) score.

```
Queries evaluated: 256
Recall@10: 0.8516
MRR@10 (after rerank): 0.7468
Bi-Encoder CosSim: 0.3510
```

```
Queries evaluated: 256
Recall@10: 0.8516
MRR@10 (after rerank): 0.7468
Bi-Encoder CosSim: 0.3395
```

```
Queries evaluated: 256
Recall@10: 0.8516
MRR@10 (after rerank): 0.7468
Bi-Encoder CosSim: 0.2761
```

Figure 5

```python
# v1
def get_inference_system_prompt() -> str:
    prompt = (
        f"You are a meticulous test-taker; you excel at question answering. "
        f"You are good at finding the right answer among many passages. "
        f"You are also good at identifying passages that may not contain the answer you want, even if they seem to be related to the question. "
    )
    return prompt
def get_inference_user_prompt(query : str, context_list : List[str]) -> str:
    passages = "\n".join(
        [f"Passage {i}: {p.strip()}" for i, p in enumerate(context_list)]
    )
    prompt = (
        f"Find the answer for the query based on the given passages. "
        f"If there's no answer in the given passages, just answer 'CANNOTANSWER'\n"
        f"query: {query}\n"
        f"{passages}"
    )
    return prompt
def parse_generated_answer(pred_ans: str) -> str:
    parsed_ans = pred_ans
    return parsed_ans
```

Figure 6

Q2: Prompt Optimization

```python
# base
def get_inference_system_prompt() -> str:
    prompt = ""
    return prompt

def get_inference_user_prompt(query : str, context_list : List[str]) -> str:
    passages = "\n".join(
        [f"Passage {i}: {p.strip()}" for i, p in enumerate(context_list)]
    )
    prompt = (
        f"query: {query}"
        f"{passages}"
    )
    return prompt

def parse_generated_answer(pred_ans: str) -> str:
    parsed_ans = pred_ans
    return parsed_ans
```

Figure 7

```python
# v2
def get_inference_system_prompt() -> str:
    prompt = (
        "You are an intelligent and precise question-answering assistant. "
        "You must answer questions based ONLY on the given passages. "
        "If the passages do not contain the answer, reply exactly with: CANNOTANSWER. "
        "Do not hallucinate or guess any information. "
        "Be concise, factual, and use formal English in your responses."
    )
    return prompt

def get_inference_user_prompt(query : str, context_list : List[str]) -> str:
    joined_context = "\n".join(
        [f"[Passage {i+1}]\n{ctx.strip()}" for i, ctx in enumerate(context_list)]
    )
    prompt = (
        f"Answer in complete sentences and rely strictly on the information from the given passages ONLY. "
        f"If the answer cannot be found in the passages, reply exactly with: CANNOTANSWER. "
        f"For example:\n"
        f"Question: Where is the capital of Taiwan?\n"
        f"[Passage 1]\nTaiwan is famous for its delicious food.\n"
        f"[Passage 2]\nThe capital of France is Paris.\n\n"
        f"[Passage 3]\nTaiwan's cities are busy.\n"
        f"[Passage 4]\nTaipei is in the northern Taiwan.\n"
        f"Answer: CANNOTANSWER\n\n"
        f"The following are passages relevant to a question. "
        f"Use ONLY these passages to answer.\n\n"
        f"Question: {query}\n"
        f"{joined_context}\n"
    )
    return prompt

def parse_generated_answer(pred_ans: str) -> str:
    ans = pred_ans.strip()
    if re.search(r"(CANNOTANSWER|cannot\s*answer|no\s*answer|unknown|not\s*sure|cannot\s*be\s*determined)", ans, re.IGNORECASE):
        return "CANNOTANSWER"
    match = re.search(r"(?i)answer[:]\s*(.*)", ans)
    if match:
        return match.group(1).strip()
    lines = [line.strip() for line in ans.split("\n") if line.strip()]
    if len(lines) > 0:
        return lines[-1]
    return ans
```

Figure 8

Q3: Additional Analysis

- As shown in Figure 5, the Bi-Encoder CosSim score differs with different prompts, but the Recall@10 and MRR@10 are completely the same, which may indicate the prompts tested didn't affect the performance of the retriever and the reranker.
- When tested on the first 128 examples, the base prompts, which do no more than layout the query and passages, take 8:39 to infer, whereas prompts v2, which strongly prevents hallucination, take only 40 seconds. The former is 12.975 times as long as the latter, and the Bi-Encoder CosSim score drops from 0.3418 to 0.2820 (Figure 9).

```
(adl_hw3) eva@DESKTOP-Q59RNK3:/mnt/c/ntu_csie/1141/adl/hw3/prompt_test$ python inference_batch.py --retriever_model
_path retriever_mnr_1 --reranker_model_path reranker_hard_neg_1 --test_data_path ./data/test_open.txt --test_data_s
ize 128
Retriever VRAM: 0.47 GB
len(test): 3343
Reranker VRAM: 0.13 GB
Loading checkpoint shards: 100%|                                                    | 2/2 [00:02<00:00,  1.39s/it]
LLM VRAM: 3.44 GB
100%|                                                                                | 2/2 [08:39<00:00, 259.98s/it]
Queries evaluated: 128
Recall@10: 0.8359
MRR@10 (after rerank): 0.7250
Bi-Encoder CosSim: 0.3418
(adl_hw3) eva@DESKTOP-Q59RNK3:/mnt/c/ntu_csie/1141/adl/hw3/prompt_test$ python inference_batch.py --retriever_model
_path retriever_mnr_1 --reranker_model_path reranker_hard_neg_1 --test_data_path ./data/test_open.txt --test_data_s
ize 128
Retriever VRAM: 0.47 GB
len(test): 3343
Reranker VRAM: 0.13 GB
Loading checkpoint shards: 100%|                                                    | 2/2 [00:00<00:00,  3.14it/s]
LLM VRAM: 3.44 GB
100%|                                                                                | 2/2 [00:40<00:00, 20.45s/it]
Queries evaluated: 128
Recall@10: 0.8359
MRR@10 (after rerank): 0.7250
Bi-Encoder CosSim: 0.2820
```

Figure 9

Q3: Additional Analysis

- When tested on the first 128 examples using prompts v1, using the finetuned reranker gets slightly higher MRR@10 and Bi-Encoder CosSim scores than those of the base reranker, shown in order in Figure 10.

```
(adl_hw3) eva@DESKTOP-Q59RNK3:/mnt/c/ntu_csie/1141/adl/hw3$ python inference_batch.py --retriever_model_path retrie
ver_mnr_1 --reranker_model_path reranker_hard_neg_1 --test_data_path ./data/test_open.txt --test_data_size 128
Retriever VRAM: 0.47 GB
len(test): 3343
Reranker VRAM: 0.13 GB
Loading checkpoint shards: 100%|                                                    | 2/2 [00:02<00:00,  1.07s/it]
LLM VRAM: 3.44 GB
100%|                                                                                | 2/2 [02:17<00:00, 68.57s/it]
Queries evaluated: 128
Recall@10: 0.8359
MRR@10 (after rerank): 0.7250
Bi-Encoder CosSim: 0.3585
(adl_hw3) eva@DESKTOP-Q59RNK3:/mnt/c/ntu_csie/1141/adl/hw3$ python inference_batch.py --retriever_model_path retrie
ver_mnr_1 --reranker_model_path cross-encoder/ms-marco-MiniLM-L12-v2 --test_data_path ./data/test_open.txt --test_d
ata_size 128
Retriever VRAM: 0.47 GB
len(test): 3343
Reranker VRAM: 0.13 GB
Loading checkpoint shards: 100%|                                                    | 2/2 [00:00<00:00,  2.95it/s]
LLM VRAM: 3.44 GB
100%|                                                                                | 2/2 [02:11<00:00, 65.52s/it]
Queries evaluated: 128
Recall@10: 0.8359
MRR@10 (after rerank): 0.7191
Bi-Encoder CosSim: 0.3581
```

Figure 10